
Improving Parameter-Efficiency of Particle Transformer for LHC Jet Tagging

Mykhailo Bondarenko^{*1} Anastasiia Mazur^{*1} Oleh Prostakov^{*1}

Abstract

Jet Tagging is a field of High-Energy Physics, which deals with the problem of classifying the particle origins of hadronic jets (narrow cones of particles flying in the same direction). In this paper, we report on the results of our project for the Machine Learning course, which was dedicated to advancing that field. Our main contributions are: (i) the introduction of the "Same Weights" modification, achieving state-of-the-art performance on JetClass by improving accuracy by 0.1% without increasing parameter count, and up to 0.4% when the parameter count is expanded; (ii) the proposal of the "Weighted Layer Decode" approach, further improving accuracy by up to 0.25% on the Top Tagging dataset without fine-tuning, while substantially reducing the model size to 9% of its original parameters; (iii) the adaptation of advanced optimization methods, such as the SOAP optimizer, which leads to higher accuracies; and (iv) a comprehensive evaluation of various experimental modifications, providing insights and identifying promising directions for future research. The proposed techniques do not require domain-specific inputs, making them easy to implement for existing architectures.

Project Artifact Availability:

The source code, data, and/or other artifacts have been made available at:

- <https://github.com/michael-2956/weaver-core> (our working repository for Particle Transformer)
- https://github.com/michael-2956/particle_transformer (configurations for Particle Transformer)
- <https://www.kaggle.com/code/oprostakov/jet-tagging-pipeline> (a Jupyter Notebook with our base FFN)

^{*}Equal contribution ¹Applied Sciences Faculty, Ukrainian Catholic University, Lviv, Ukraine 79026. Correspondence to: Mykhailo Bondarenko <mykhailo.bondarenko@ucu.edu.ua>, Anastasiia Mazur <mazur.pn@ucu.edu.ua>, Oleh Prostakov <prostakov.pn@ucu.edu.ua>.

- <https://github.com/idgafd/weaver-core/blob/main/weaver/nn/model/ParticleTransformer.py> (architectures of Particle Transformer with attention mask modifications)

1. Introduction

High-energy collisions at the Large Hadron Collider (LHC) result in the creation of quarks and gluons of multiple types. Further, these particles hadronise into collimated sprays, which are then detected by experiments far from the collision point. By involving a lot of different interactions, jet physics presents a great opportunity for carrying on Standard Model measurements and New Physics searches. However, the modern equipment allows to only detect the results of the hadronization process.

Jets initiated by different elementary particles exhibit similar characteristics, however for many analyses conducted in High Energy Physics (HEP) it's crucial to understand which type of particle initiates it. Therefore, the study of jet tagging has been more and more reliant on machine learning, which significantly improved performance (Qu et al., 2024; Kasieczka et al., 2019).

In this paper, we cover our work performed in the scope of the project for our Machine Learning course. We started with a simple fully connected neural network to obtain a reasonable baseline, tried some improvements until we reached the saturation point of that model, and then moved on to the SoTA architectures in the field of jet tagging, notably, the Particle Transformer architecture. While working with these SoTA models, we performed numerous experiments in an attempt to achieve new highs in terms of accuracy, training and inference time, and the model size.

The main contribution of our work to the field of jet tagging lies in the improved Particle Transformer architectures, which achieve **+0.25%** better accuracy on the Top Tagging (Kasieczka et al., 2019) dataset without fine-tuning while decreasing parameter count by **91%**, and up to **+0.1%** better accuracy on JetClass (Qu et al., 2024) while keeping the same parameter count. The **+0.1%** increase remains consistent when the architecture is scaled up, reaching **86.46%** in total, which is **+0.36%** from the original baseline. Furthermore, this paper describes a set of experiments that did not yield significant improvements to the model, and can

act as either the confirmation of similar results received by different teams at CERN, or the indicators of the research directions that might be less fruitful than the other ones.

2. Related Works

2.1. Top Quark Tagging Reference Dataset

The dataset proposed by (Kasieczka et al., 2019) consists of 2M labeled jets of 2 types – top-quarks ($t \rightarrow bqq'$) and QCD background (quarks and gluons, q/g). The input features provide only the kinematic information about particles, specifically their momentum components p_x, p_y, p_z and energy E , without any additional particle identification or interaction details. This dataset was used as a benchmark throughout our experiments on all models due to its relatively small size and simplicity. In this work, we will be referring to this dataset as TopT.

2.2. Particle Transformer for Jet Tagging

Particle Transformer, or ParT, is a transformer model proposed by (Qu et al., 2024), which currently holds SoTA results in the field of jet tagging by the utilization of the Attention mechanism. This model was used as the main subject in most of our experiments, where we aimed to improve it by manipulating the particle interaction matrix, enabling weight sharing, incorporating a MoE blocks, etc.

In addition, the same work proposes a new reference dataset - JetClass. The dataset consists of 100M labeled jets, contains 10 tagging classes, and provides various new information about particles in addition to the kinematic data. This extended dataset was used at the end of our experiments to explore how well our new approaches scale with data.

3. Strategies

3.1. Base Feed-Forward Network

To get a reasonable baseline, we started with a simple feed-forward neural network. The first version of our network consists of 4 blocks, followed by a softmax function. Each block is composed of a fully-connected layer of 200 neurons, followed by the ReLU activation function. Overall, the model ended up having $\approx 2 \cdot 10^5$ parameters.

This base network was trained on the TopT dataset for 20 epochs, using the cross-entropy loss and Adam optimizer, and it resulted in the accuracy of $\approx 80\%$ on the test subset.

To improve performance, the following modifications were made:

- The number of blocks in the model was set to 8.
- The number of neurons of all layers was set to 256.

- Dropout layers were inserted at the end of each block.
- Batch normalization layers were introduced after the fully-connected layers in each block.
- The number of training epochs was increased from 20 to 100, keeping the same flat learning rate. The decision was made based on the fact that overfitting was never noticed based on validation results, and the accuracy kept slowly climbing throughout the training.
- Residual connections were added between blocks. In addition, the output of the first block was propagated to the activation function of the very last block.
- ReLU function was replaced with SiLU throughout the model.
- Adam optimizer was replaced with AdamW
- Cosine Annealing scheduler was added to dynamically adjust the learning rate of the optimizer.
- Convolution was added as the first layer to enforce invariance in respect to the order of particles.

With all these improvements, we ended up with a model consisting of $\approx 6 \cdot 10^5$ parameters, trained over 100 epochs. This final model managed to score accuracy of 89% on the test set, significantly outperforming our starting version.

3.2. MoE for Particle Transformer

Mixture of Experts is a form of ensemble learning, where a single feedforward network is replaced by n experts and a router, which passes every input vector to k , $k < n$ experts best suited to process it, and aggregates their results. Although the technique is relatively novel, it has been used extensively in various transformer models, most notably, the recent DeepSeek-V2 (DeepSeek-AI et al., 2024), and, reportedly, GPT-4.

While being promising, MoE also has a number of drawbacks, which prevent its incorporation into more mainstream models. First, MoE models consume a lot of VRAM. Although the input data only goes through a number of experts, keeping the rest 'dormant', all the experts must be loaded into memory. Second, MoE models are notably slower than the regular ones. Modern deep learning is built on GPUs, which are highly optimized to perform batched parallelized GEMM operations. In contrast, experts typically process data sequentially with respect to one another, which can slow down the whole training process in 2-3 times. Finally, during the training, the model would often try to exploit a small subset of experts for almost all inputs, basically, converging MoE to the original FFN.

In Particle Transformer, we replaced the feedforward part of both regular and class attention blocks with a MoE module. With this approach, we allow the router to choose the best subset of experts for each token coming out of the attention layer.

Our first implementation was based on the MoE architecture used in Switch Transformers (Fedus et al., 2022). In this setup, we started with 16 experts, each with dimensions of the original FFN, where 2 would be called for each token. The router would rearrange the incoming batch of tokens in a way to make each expert process their set of tokens in a batch, all at once. Moreover, a dedicated loss function was incorporated to penalize the model for exploiting a subset of experts and incentivize it to aim for a roughly uniform distribution of tokens for the experts. Unfortunately, the results were somewhat underwhelming. The training became 5 times slower, while the model did not manage to achieve significantly better results.

With the unsuccessful first attempt, we decided to incorporate the techniques proposed in the DeepSeek-V2 paper into our model. Most notably, we ended up with 32 fine-grained experts, each with half of the dimensions of the original FFN, where 3 would be called for each token. Additionally, we added a single shared expert into the pool, which is called for every single token, and is there to capture the ‘common knowledge’ about the problem, letting other experts specialize on the peculiarities of their subsets of tokens. Finally, we replaced the expert-level loss with the one described in the paper. As a result, we manage to accelerate the training to just 3 times of the original MoE implementation, and, in combination with the “same weights” modification, slightly improve the results on the validation set - **94.175%** vs **94.05%** of the original model on the TopT dataset, using the SOAP optimizer described later. Unfortunately, we did not have much time for experiments with MoE, so there is a lot of space for improvement with basic parameter tuning.

3.3. The “Same Weights” Modification

The Same Weights modification (or **samewts**, for short) makes all the blocks within the Particle Attention group and Class Attention group share weights (the blocks do not share weights between groups). This modification originates from the idea that the model tries to reconstruct the physics of particle interaction back within those blocks, from the point where the jet’s constituents were recorded by the detectors to the origin vertex of the jet. The model attempts to recreate particle interactions between those two events. Based on the hypothesis that the underlying physics stays the same through time, it makes sense to try teaching a single Particle Attention and a single Class Attention block those laws, and then rewind the process by applying them multiple times.

Basically, when the **samewts** modification is applied, in-

stead of going through N different Particle Attention and K different Class Attention blocks sequentially, the input tensor makes N passes through a single Particle Attention block, and then K passes through a single Class Attention block. This process is illustrated on Figure 1.

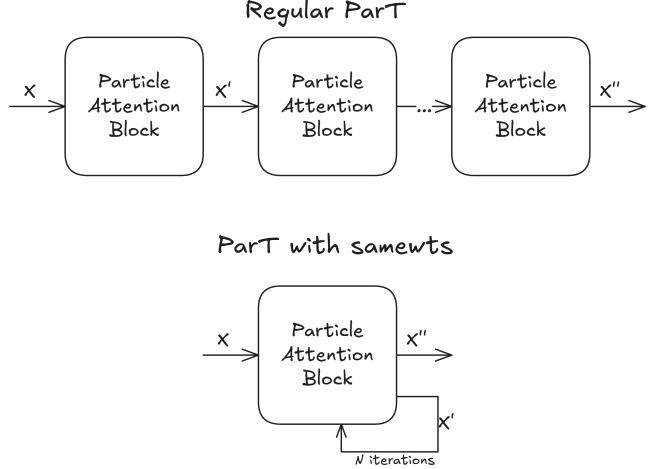


Figure 1. The “Same Weights” modification

In practice, the **samewts** modification also provides regularization for the model, which especially benefits when training on smaller datasets, such as TopT. Moreover, it reduces the number of parameters significantly, which makes the training faster, accelerating the training process. As a result, we achieve an accuracy of **94.20%** on TopT with the SOAP optimizer without fine-tuning, while needing only 544K parameters, which is only 25% of the original ParT architecture.

3.4. Weighted Layer Decode modification

After isualising the resulting attention weights after implementing the “same weights” modification, we observed that different jets require different number of Particle Attention blocks to be processed. Some of them are simple, and would require only a few blocks to make a reasonable verdict. Some are complicated, and require the model to investigate particle interactions within the jet for longer to make a guess.

In order to classify highly structured complex jets, one might consider to increase the number of Particle Attention blocks. However, such an approach might degrade performance on simple jets. Applying only several blocks might already give the model enough information to accurately classify the jet, and pushing this output through many following layers is likely to scramble this information. Moreover, it is plausible that interactions investigated by some blocks might be more important than others.

To solve the problems outlined above, we propose the modification called Weighted Layer Decode (**WLD**). Instead of

passing only the output of the last Particle Attention (encoder) block to the Class Attention (decoder) blocks, we forward the output of each encoder block to its separate decoder, which all share weights, and then do a weighted aggregation on the Class Attention outputs to obtain the result. This process is illustrated by Figure 2. Such an approach enables us to put more emphasis on the outputs of certain Particle Attention blocks, which allowed us to improve per parameter accuracy for the price of worse performance. As a result, we achieve a score of **94.25%** on TopT without fine-tuning, while needing only 197K parameters – only 9% of the original model.

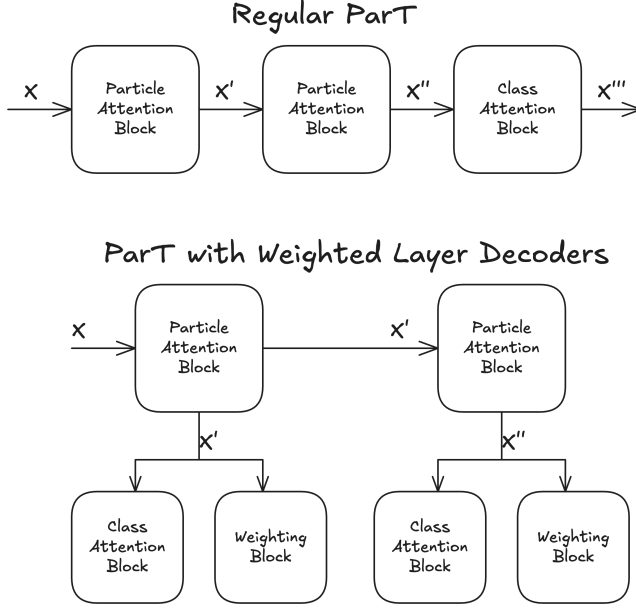


Figure 2. WLD modification

3.5. Attention Mask Modifications

The original Particle Transformer computes attention biases through learnable projections of Lorentz-invariant pairwise features, including angular distances ΔR , relative transverse momentum k_t , and invariant masses. While this approach already incorporates physics knowledge, we explored three alternative strategies for computing attention maps to potentially improve computational efficiency or interpretability.

ParticleTransformerModU replaces the learnable pairwise embedding with direct projection of hardcoded physics features: $\mathbf{u}_{ij} = [\Delta\eta_{ij}, \sin(\Delta\phi_{ij}), \cos(\Delta\phi_{ij}), \Delta R_{ij}^2, r_{ij}, s_{ij}, d_{ij}]$, where r_{ij} encodes p_T ranking and s_{ij} represents octant similarity. This approach eliminates the need for learning pairwise representations but proved significantly less effective than the baseline. **ParticleTransformerModUHybrid** combines both strategies through weighted fusion: $\text{Attention}_{ij} = \alpha \cdot \text{PairEmbed}(\mathbf{v}_i, \mathbf{v}_j) + (1 - \alpha) \cdot \text{PhysicsProj}(\mathbf{u}_{ij})$, where α is learnable. It achieved the best performance among

our modifications but still slightly underperformed the original architecture. And the **ParticleTransformerModU-Layerwise** implements layer-specific physics projections with an expanded 12-dimensional feature set, allowing different transformer layers to specialize in different physics aspects, but this approach suffered from excessive memory consumption due to layer-specific parameter multiplication.

Experimental results consistently favored the original learnable approach and suggests that end-to-end learning of pairwise representations captures particle relationship complexities more effectively than explicit feature engineering, making further architectural modifications in this direction unpromising without more sophisticated integration strategies. Potentially, rather than simply adding features to the existing attention maps, successful improvements would likely require entirely different approaches. These could include architectures that handle different physics scales separately (particles, jets, and whole events), regularization techniques that encourage the model to respect physics laws without forcing specific features, or attention mechanisms that adapt based on the type of physics event being processed.

4. Experiments & Results

4.1. Testing the impact of model size with the TopT dataset

In this series of experiments, we will test how various dimensions of the ParticleTransformer architecture affect its validation set predictive ability, overfitting, and other training characteristics like speed of learning.

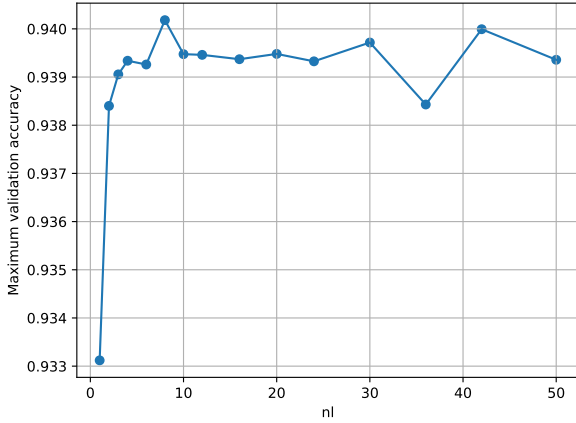
To do this, we split the model size into the following 3 dimensions:

- **Length.** This parameter is responsible for the number of layers in a given network. As the initial ParT architecture has 8 processing and 2 classification layers, this means that the length parameter for the initial ParT is $l = 10$, with $c_l = 0.2$ being the fraction of classification layers. To recover their number, we will use $c = \lceil c_l \cdot l \rceil$. We will vary l from 1 to 50, taking increasing steps such that $l \in \{1, 2, 3, 4, 6, 8, 10, 12, 16, 20, 24, 30, 36, 42, 50\}$.

On the Top Tagging dataset, the best results were achieved with the number of layers set to 8. Anything smaller than that causes the model to underfit, and anything higher - to overfit, although we noticed that the effects of overfitting were limited due to limited training time.

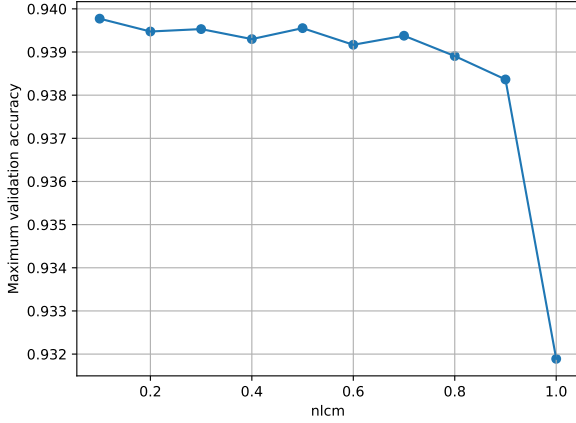
The results for all variations of the number of layers (**nl**) are demonstrated in Figure 3.

- **Fraction of classification layers.** While keeping $l =$


 Figure 3. Relation between Accuracy and **nl**

10, we will vary c_l such that $1 \leq c \leq 10$, where the upper bound requires the class token to appear at the very beginning.

Our results demonstrate that the multiplier of the number of classification layers (**nlem**) does not have a significant effect on accuracy, unless it takes an extreme upper value (Figure 4), where there are no encoders left at all. Then, the model starts overfitting, and the accuracy plummets. The best value turns out to be $c_l = 0.1$.

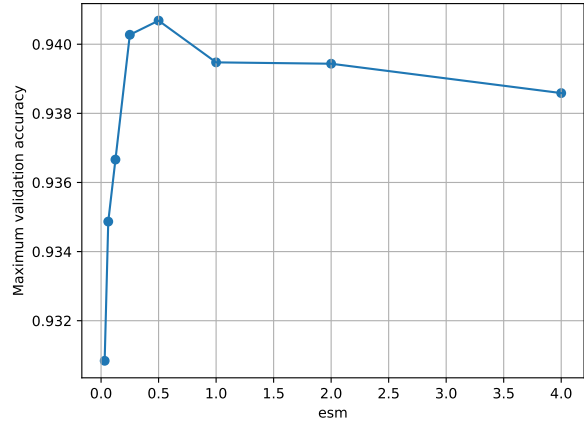

 Figure 4. Relation between Accuracy and **nlem**

- **Embedding dimension.** In the ParT architecture, before the input particles enter the attention blocks, they are first embedded by an FCN comprising of 3 linear transformations, with dimensions [128, 512, 128]. These dimensions may also be scaled by a scale factor $s_e \in \{\frac{1}{32}, \frac{1}{16}, \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1, 2, 4\}$. The number of heads for the default setting is 8, which means 16 neurons per head for 128 embedding size. We try to scale the

number of heads to keep the number of neurons per head the same throughout s_e values, reducing it to 4 and 8 for settings $\frac{1}{32}$ and $\frac{1}{16}$, where it is impossible to obtain 16 neurons per head.

The results achieved on the Top Tagging dataset demonstrate that the best scaling of the embedding dimension (**esm**) is $s_e = \frac{1}{2}$, with other scales negatively affecting accuracy, regardless of whether we scale the dimensions up or down. We believe that scaling down causes the model to underfit, and scaling up - to overfit the data.

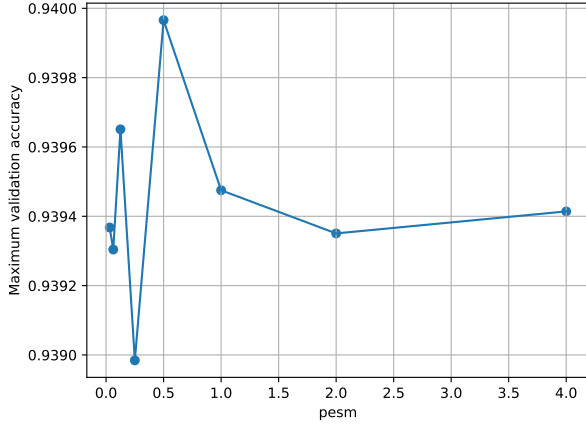
The relation between Accuracy and Embedding Scale Multiplier (**esm**) is demonstrated in Figure 5.


 Figure 5. Relation between Accuracy and **esm**

- **Pair embedding dimension.** The particle embeddings are not the only embeddings in the ParT architecture. In fact, key to the accuracy of ParticleTransformer is the pair embeddings, which are added to the attention mask of each head before it is applied to obtain output values. To obtain those, pairwise features of each particles are passed through a series of 1x1 convolutions, having [64, 64, 64] as their default dimensions. Separately from s_e , we scale those by $s_p \in \{\frac{1}{32}, \frac{1}{16}, \frac{1}{8}, \frac{1}{4}, \frac{1}{2}, 1, 2, 4\}$.

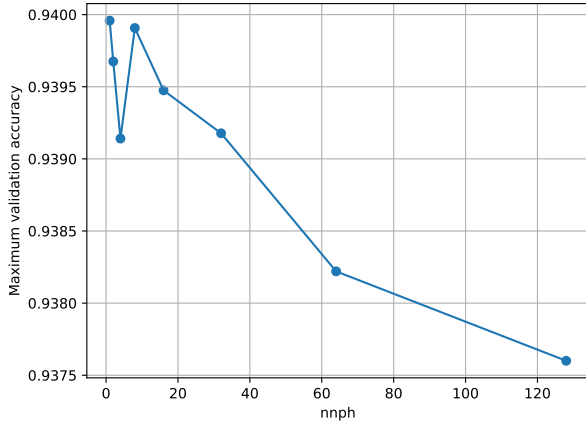
Contrary to what we saw for s_e , pair embedding scale multiplier (**pesm**) does not seem to have any significant impact on accuracy. We observe that even when we scale the convolutions to have [2, 2, 2] neurons, the results appear within statistical error from one another.

- **Number of neurons per head.** As discussed previously, the ParT architecture has 16 neurons per head by default, which means 8 heads for 128 embedding size. Since the additive U matrix is different per every head, the number of heads may influence the number of different additive attention masks, and as such


 Figure 6. Relation between Accuracy and **pesm**

affect the accuracy. To test this hypothesis, we conduct experiments by varying the number of neurons $n_h = \{1, 2, 4, 8, 16, 32, 64, 128\}$, which corresponds to a number of heads ranging from 1 to 128.

The best results of 93.99% accuracy were achieved with the number of neurons per head (**nnph**) equal to 1. We hypothesize that this happens since higher n_h values equate to decreasing the number of U matrices, and lower values make the number of U matrices large, which improves the accuracy. From this we make the conclusion that for U matrices to provide measurable benefit it is important that there is a large amount of them. We notice also that $n_h = 1$, which equates to 128 U matrices, has a similar accuracy to $n_h = 8$, which has 16 U matrices. Results for all parameter values are demonstrated in Figure 7.


 Figure 7. Relation between Accuracy and **nnph**

4.2. The "Same Weights" Modification

To test whether the "Same Weights" modification to the architecture actually performs better than the base architecture and not just perform better by the merit of having less parameters and thus overfitting less, we retrained the ParT model with this modification enabled for every embedding-scale value $s_e \in \{0.0625, \dots, 4\}$. To compare the models, we plotted correlation between parameter count and accuracy on Figure 8. As we can see, the new architecture, except when the parameter count nears 30M, lies strictly above the blue baseline: for an equal parameter number the **ParT-samewts** variant yields higher test accuracy per parameter count. In addition, we found that during training it converges faster and fits into a smaller memory footprint than base **ParT**.

For this modification, all experiments use the SOAP optimiser with the same learning-rate schedule as the original ParT implementation, and all other hyper-parameters were kept fixed.

4.3. The "Weighted Layer Decode" Modification

Enabling the **WLD** modification on top of **samewts** gives the green curve on Figure 8. Although the parameter count remains nearly unchanged, the **WLD** variant consistently provides the best accuracy-to-size trade-off across scaling regimes that fall in the 60K to 300K parameter count range, surpassing both the default architecture and the **samewts** modification.

For this modification, all experiments use the SOAP optimiser with the same learning-rate schedule as the original ParT implementation, 28 Particle-Attention encoder blocks followed by 2 Class-Attention decoder blocks, and identical data preprocessing and augmentation settings. We found that increasing the number of encoder blocks improved accuracy, but at high numbers training became unstable and used up a lot of VRAM. All other hyper-parameters are kept fixed.

4.4. The "Mirror" Effect

As a model for binary classification, ParT outputs values on the scale of 0 to 1. Those values can be interpreted as a measure of confidence of the model that the input jet was produced by the top quark, where values close to 0 mean that the model is 0% confident that the particle was a top quark (i.e. 100% confident that it is part of QCD background), and values close to 1 mean that the model is 100% confident that the particle is a top quark.

An obvious quality of a well-fit model is that it makes very few confidently-incorrect predictions by giving $\approx 0\%$ confidence for actual top quarks, or $\approx 100\%$ for other particles.

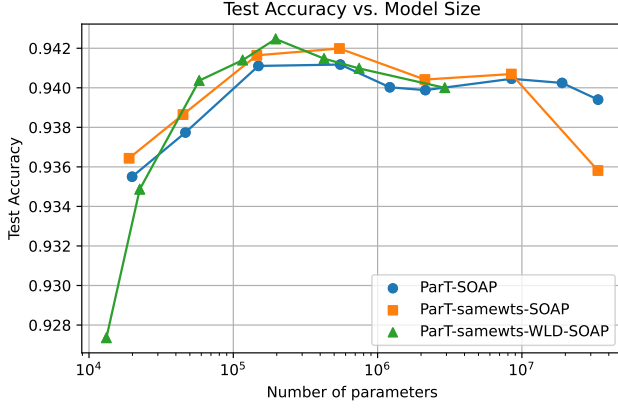


Figure 8. Comparative test performance of ParT variants by parameter count

However, when doing EDA of the predictions by ParT on the TopT dataset, we noticed an interesting phenomenon.

First, we plotted the distribution of the probability of ground truth (p_{gt}) on a log scale for Y , as you can see on Figure 9. In our context, p_{gt} of 1 means that the model made a confidently-correct prediction – 1 for an actual label or 0 for the incorrect label. And p_{gt} of 0 refers to the opposite scenario – a confidently-incorrect prediction.

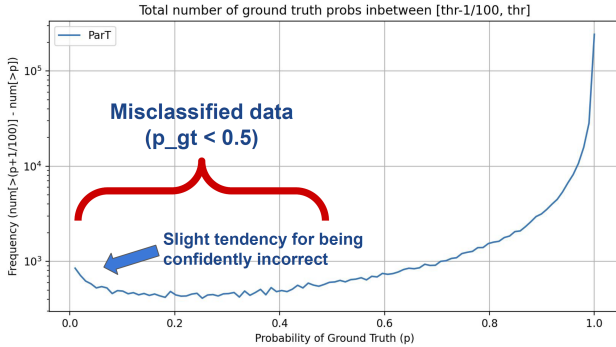


Figure 9. Probability of ground truth (log-scale for Y)

Take notice that although we expect confidently-incorrect predictions to be extremely rare, there is actually a bump on the left of the picture. Our interpretation for this bump is that the dataset contains a small portion of jets for class 1 that are basically indistinguishable from the jets for class 0, and vice-versa. We called those particles **mirrored**, which gives the name to the effect we describe here.

To confirm this effect, we took a portion of the data that was misclassified by the model – namely the one where $p_{gt} < 0.5$, capturing $\approx 6\%$ of the data, and trained a scaled-down version of ParT called ParT-Small ($s_e = 0.25$, $s_p = 0.5$, $l = 8$, $c = 2$), resulting in an accuracy of 78.839% on the training ($p_{gt} < 0.5$) subset. If the hypothesis is correct, the accuracy on the opposite set ($p_{gt} \geq 0.5$), spanning the rest ($\approx 94\%$) of the data, must be very low, as these examples

are supposed to be identical but have opposite labels. Indeed, the accuracy turns out to be as low as 2.061%, which serves as a pretty solid proof of our hypothesis.

Finally, if we use ParT-Small to predict the entire dataset, and then take the misclassified subset as before (this time spanning 93.227% of the data), we can use this subset to train another ParT-Small architecture. After repeating this process recursively 12 times, we observe that the accuracy on the opposite split decreases, while the accuracy on the training split increases, reaching 0.94% (on $\approx 92.43\%$ of the data) and 96.4% (on $\approx 7.57\%$ of the data) respectively on iteration 12, as illustrated on Figure 10. Note that during this process, the class balance (top quarks vs. QCD) remains relatively equal on both sets, meaning that we can attribute this effect to “mirror” jets in the data.

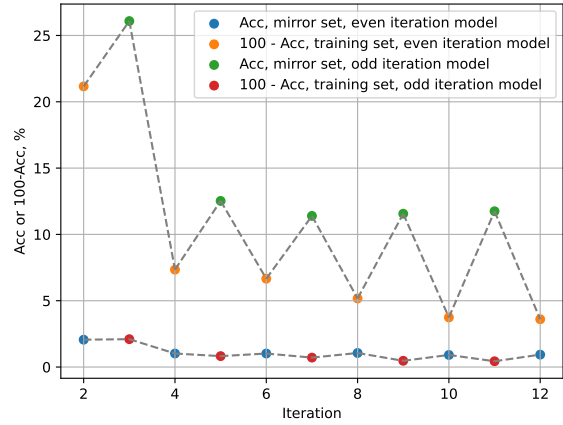


Figure 10. Results of recursively training models on the mirrored set of jets. Here, “mirror” refers to the subset of jets misclassified by the previous model. Two punctuated lines connect the subsets that share the majority of the jets. The figure shows that the accuracy slowly increases on the training set, while decreasing on the mirror set for each model.

Based on that discovery, we decided to try to train a separate model, which would be capable of distinguishing mirrored particles from non-mirrored. This is discussed in more detail in Subsection 4.7.4.

4.5. Optimizers

The authors of ParticleTransformers rely on Ranger (Wright & Demeure, 2021) as the default optimizer of their choice. However, since this optimizer was released over 4 years ago, and the popularity of transformer models has surged since then, we had a theory that some better optimizers could have been found.

In our experiments, we achieved the accuracy of 93.8% on the **validation** set with Ranger. This value will be used as a baseline going forward in this section.

4.5.1. ADAMW

The best results that we obtained for the base FFN were achieved with AdamW, which made it an obvious first candidate. Unfortunately, it did not manage to meet our expectations. No matter how long we would spend tuning its hyperparameters, it would never outperform Ranger.

4.5.2. LION

Lion (Chen et al., 2023) was discovered at Google Brain in 2023, and according to the information found online, often outperformed Adam and AdamW, which made it our next candidate. Almost immediately, we managed to achieve validation loss similarly to what we saw for Ranger. After spending some time tuning the hyperparameters, we managed to score 93.911% on the validation set, getting ahead of our baseline. Additionally, although the number of iterations it took the loss to converge stayed the same, Lion would execute those iterations marginally (10 – 15%) faster.

4.5.3. SHAMPOO

After achieving success with Lion, we decided to give a try to the second-order optimizers. The main difference between them and the optimizers we discussed before lies in the fact that they incorporate Hessians in addition to the gradients when minimizing the loss function. Obviously, given the size of modern networks, those Hessians are not computed directly, but approximated with some clever tricks. Still, the incorporation of Hessian makes them consume more VRAM and take longer to run when compared to the first-order optimizers.

The first second-order optimizer that we tried is Shampoo (Gupta et al., 2018). It looked promising on paper; however, we also noticed that it was often described as slow. After starting our experiments, we quickly confirmed this slowness. The experiments were shut down less than half an hour after they started, since it took the optimizer more than 20 minutes to compute the first preconditioned Hessian approximation (granted, that it would stay cached for a number of iterations), despite running on the H100 GPU.

4.5.4. SOAP

In 2024, the research team at Harvard released a paper about the SOAP (Vyas et al., 2025) algorithm - a modification of Shampoo (yes, all the names are real), promising to be much faster and more stable than its predecessor, which made it the logical next step in our experiments. With it, we managed to hit the accuracy of 94.05%, and achieve the convergence in just half the number of epochs of Ranger. Granted that each epoch took roughly 40% more time than for Ranger, it was still a massive success in terms of both accuracy and training speed.

4.5.5. CONSLUSIONS

To conclude our experiments with optimizers, we discovered that Lion and SOAP work better than the default Ranger (93.911% and 94.05% vs 93.8% respectively). Both of these newer optimizers also run faster, and while SOAP requires marginally more VRAM, Lion consumes about as much resources as Ranger.

4.6. JetClass experiments

We tested various model variations on the JetClass dataset. The results of the experiments that were completed at the time of writing the report are present in Table 1.

We tested the following settings: enabling the "same weights" modification, changing the s_e setting, selecting an optimizer (Ranger, SOAP) and varying the decay fraction, which is responsible for the fraction of training time that the exponential decay takes up after the initial flat learning rate period (0.3 in the original setup). All other hyperparameters are kept the same as in the original ParT architecture.

In addition, although the WLD modification remains untested due to submission deadline, it shows promising results during training: it achieves a validation accuracy of 86.08% on epoch 29, while the baseline with a similar parameter count achieved only 85.05%, and the "same weights" modification with the same optimizer/scheduler/parameter count achieved 85.90%. The learning curve of this midification corresponds to that of "Part-samewts-esm4-SOAP-lrd07", meaning that it has potential to achieve an accuracy of up to 86.46% on the test set.

Notably, we achieve an accuracy of 86.19% using the samewts modification of the architecture, $s_e = 2$, the SOAP optimizer and a decay fraction of 0.7, surpassing the original results reported for ParticleTransformer (Qu et al., 2024) by 0.09%, and by 0.13% when compared to the replicated run, while very slightly decreasing the number of parameters. In addition, when scaled up, the advantage remains, reaching an accuracy of 86.46% in total, which is +0.36% from the reported baseline and +0.09% from the original architecture scaled using $s_e = 2$ and trained with the same optimizer and scheduler. This highlights the advantage of the "same weights" modification.

4.7. Smaller experiments that did not improve results

This section is dedicated to various attempts at improving the model that did not yield any benefits.

4.7.1. SCALABLE SOFTMAX

Scalable Softmax (Nakanishi, 2025) was used for the attention layers in LLama-V4, and since it did not require a lot of effort to be added to Particle Transformer, we decided to

Model Name	Test Acc.	#Params (M/k)	Comp. Complexity	Same- wts	ESM Val	Opti- mizer	Decay Frac.
ParT-samewts-ranger	85.69	545.98 k	306.18 MMac	✓	1	ranger	0.3
ParT-samewts-SOAP	85.71	545.98 k	306.18 MMac	✓	1	soap	0.3
ParT-esm05-SOAP-lrd07	85.62	552.01 k	133.65 MMac	✗	0.5	soap	0.7
ParT-samewts-esm2-ranger	86.08	2.13 M	991.55 MMac	✓	2	ranger	0.3
ParT-samewts-esm2-SOAP-lrd07	86.19	2.13 M	991.55 MMac	✓	2	soap	0.7
ParT-ranger (baseline)	86.06	2.14 M	306.18 MMac	✗	1	ranger	0.3
ParT-SOAP	86.10	2.14 M	306.18 MMac	✗	1	soap	0.3
ParT-samewts-esm4-ranger	86.35	8.45 M	3.27 GMac	✓	4	ranger	0.3
ParT-samewts-esm4-nnph4-ranger	86.36	8.45 M	3.78 GMac	✓	4	ranger	0.3
ParT-samewts-esm4-SOAP-lrd07	86.46	8.45 M	3.27 GMac	✓	4	soap	0.7
ParT-esm2-SOAP-lrd07	86.37	8.47 M	991.22 MMac	✗	2	soap	0.7

Table 1. Summary of test accuracy and architectural details across models tested on JetClass. The last column denotes the fraction of training time that the exponential decay captured after the initial flat lr.

give it a try. Unfortunately, this change did not yield any significant improvements in accuracy.

4.7.2. MODEL COMPILE

One of the first things that we attempted to increase the training speed was to compile the model with torch-compile. If done correctly, the compilation can make the model 2-3 times faster. Unfortunately, there are a lot of caveats in this process. For instance, compilation does not like tensors of changing dimensions, conditional statements, optional function parameters, and so on. Given the complexity of the repository we worked with, our attempts at making the model compilable failed, wasting a few evenings in the process.

4.7.3. TRANSFORMERS WITHOUT NORMALIZATION

In March of 2025, Meta released a paper (Zhu et al., 2025) where they claimed that they managed to achieve the redundancy of normalization layers (and a few other positive effects) by replacing the activation function of their transformer models with $\tanh$. We attempted to do something similar for Particle Transformer, but the endeavor failed to bring any significant results, and in practice made the model worse.

4.7.4. MIRRORED PARTICLE CLASSIFIER

The SoTA architecture for distinguishing one type of particle from another is Particle Transformer. In this report, we focused on it in the context of classifying top quarks, however, when tasked with classifying mirrored particles, it was our default choice as well.

The major difference in classifying top quarks and mirrored particles lies in the data distribution. Whereas for the top quarks the classes are of almost equal size, there are only

$\approx 10\%$ of mirrored particles in the dataset. To battle this problem, we attempted two different approaches - reweighting the samples in the cross-entropy loss, and scrapping the CE loss altogether in favor of the focal loss.

Unfortunately, the experiments failed to bring positive results. When combining the mirror-tagging model with the regular Particle Transformer, we kept ending up with accuracy below 94%.

As a prospect for future work, we propose providing mirror/non-mirror labels to the model as an additional training signal during training.

5. Conclusions and future work

In the scope of our capstone project for the ML course, we familiarized ourselves with the field of High-Energy Physics, and investigated various machine learning models for the problem of jet tagging - from basic neural networks to CNNs, LSTMs and state-of-the-art transformers. During this half a year, we learned dozens of approaches that might (or might not :)) make the models better. From something as basic as dropout or normalization layers, to very advanced or uncommon tricks, such as MoE modules or reusing the same weights for a number of blocks of the model. Moreover, we got significantly better at conducting the research by finding, analyzing, and implementing approaches from dozens of papers and technical reports; and managing experiments by running hundreds of training sessions, aggregating and analyzing their results.

In terms of practical deliverables, our work resulted in a set of notable achievements. First, we managed to break SoTA on the JetClass dataset by **0.4%** (when compared to the replicated run), which translates to **3% less** misclassified particles, and by **0.25%** on TopT (without fine-tuning), which translates to **4% less** misclassified particles. What

makes these results even more impressive is that the model that broke the record on TopT has just **9%** parameters of the original ParT. Additionally, we documented a number of approaches from the leading research papers that did not manage to improve the model, saving time and efforts for other researchers of the field of jet tagging.

Based on the results of our research, the most promising directions for the future work are speeding up the WLD modification to allow for less computation time, as it performed best for its parameter count but required the highest computation time; improving the Mixture of Experts module; fine-tuning the "same weights" and WLD modification on TopT to compare the fine-tuned accuracy to the baseline; experimenting with more combinations of scaling parameters and modifications proposed in this paper on the JetClass dataset; building a successful mirror-tagging model or utilizing the mirror effect for accuracy improvements. Additionally, there is always space for testing more optimizers, although the returns in this direction have already been diminished. Finally, we admit that the compiled ParT would result in a significant quality-of-life improvement for future researchers, but, unfortunately, we failed with this task, and hope to do this in the future.

6. Contributions of each project member

6.1. Oleh Prostakov

- Prepared the original pipeline (feature extraction, export of results, etc) for the base FFN network.
- Investigated different optimizers/schedulers/activation functions for the base FFN network.
- Conducted experiments with optimizers for ParT.
- Performed EDA over the result of ParT, discovering the groups of jets for which the model often struggles.
- Attempted to make the ParT model compilable.
- Incorporated Scalable Softmax.
- Experimented with *tanh* instead of *GeLU*.
- Was responsible for MoE

6.2. Mykhailo Bondarenko

- Helped with the original pipeline, got a hold of ParT code.
- Implemented various changes to the ParT architecture:
 - architecture scalability in 4 directions.
 - multiple U matrices for each layer with 4 modes: independent Us, independent Us with previous

layer attn. weight input, independent Us with previous layer attn. weight input and shared weights, independent Us with current attn. weight input and shared weights employing skip connections, which were then used as new attn weights instead of being added to them. All of these techniques didn't yield results so were skipped in the report.

- came up with the "same weights" idea, which was the first architecture change that yielded improvements.
 - added the sink token, which didn't yield improvements.
 - implemented the "weighted layer decode" (WLD) feature, which was the second architecture change that improved results. Various modes for WLD weight normalization were added: softmax, Gumbel-softmax, Gumbel-softmax-hard, normalized-sigmoid, normalized-Gumbel-sigmoid. Also, two decoding modes were implemented: result aggregation (which made it here) and score aggregation (which performed worse).
 - experimented with randomizing the number of encoders in each batch in hopes that this will regularize the WLD models.
 - Added the alpha parameter to regulate the mixing between the U matrix and the attention weights. Two modes were implemented: static-alpha and decode-alpha, where the latter used a decoder to find the alpha value for the specific layer.
 - Expanded the trim module to be able to run models on a particle subset to speed up testing.
- Optimized stock attention to run faster.
 - Added linear attention to the pipeline.
 - Used linear attention for a new architecture that attempted to pairwise features (with a maximum of 128^2 tokens). This architecture had some promising results but was too unstable so was dropped.
 - Implemented various pipelines for visualizing attention masks, plotting model performance, learning curves, etc.
 - Discovered and investigated the mirror effect, coming up with various visualizations, succeeding to utilize it to obtain better training scores. Some of these techniques didn't make it to the report due to lack of time but are subject to future work.
 - In total, launched more than 200 training runs spanning more than 2000 hours of training time, and costing around \$5000 to Kaggle, Oleh and the CERN cluster.

6.3. Anastasiia Mazur

- Adapted the DeepJet model for the Top Quark Tagging Dataset using a single-stream CNN+LSTM architecture.
- Designed and evaluated multiple lightweight DeepJet-inspired architectures with different backbones.
- Introduced local attention via depthwise convolutions and explored hybrid architectures combining local context and global attention.
- Implemented custom attention bias using pairwise distances and engineered geometric features (e.g., $\Delta\eta$, $\Delta\phi$, ΔR^2 , rank, same-octant).
- Conducted architectural ablation experiments on smaller custom models, including tests with positional encodings, normalization schemes, dropout rates, activation functions, and ensemble strategies.
- Conducted experiments with custom attention masks to enrich the Particle Transformer’s attention bias:
 - Constructed a fixed attention bias using only geometric and physics-inspired features.
 - Combined the original learnable attention map with a precomputed fixed physics-based mask via a learnable interpolation parameter.
 - Implemented per-layer attention projections from 12 engineered pairwise features, with learnable temperature scaling for each attention head.

Acknowledgements

We appreciate **Pavlo Kashko** for his mentorship and the rest of the research team at CERN CMS for their support throughout the project. We appreciate CERN for allocating the computing resources required to run our experiments on the JetClass dataset. We appreciate the teaching team of the Machine Learning course at UCU for making this project possible.

Impact Statement

This paper presents work whose goal is to advance the jet tagging direction of High-Energy Physics. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Liu, Y., Pham, H., Dong, X., Luong, T., Hsieh, C.-J., Lu, Y., and Le, Q. V. Symbolic discovery of optimization algorithms, 2023. URL <https://arxiv.org/abs/2302.06675>.

DeepSeek-AI, Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Deng, C., Ruan, C., Dai, D., Guo, D., Yang, D., Chen, D., Ji, D., Li, E., Lin, F., Luo, F., Hao, G., Chen, G., Li, G., Zhang, H., Xu, H., Yang, H., Zhang, H., Ding, H., Xin, H., Gao, H., Li, H., Qu, H., Cai, J. L., Liang, J., Guo, J., Ni, J., Li, J., Chen, J., Yuan, J., Qiu, J., Song, J., Dong, K., Gao, K., Guan, K., Wang, L., Zhang, L., Xu, L., Xia, L., Zhao, L., Zhang, L., Li, M., Wang, M., Zhang, M., Zhang, M., Tang, M., Li, M., Tian, N., Huang, P., Wang, P., Zhang, P., Zhu, Q., Chen, Q., Du, Q., Chen, R. J., Jin, R. L., Ge, R., Pan, R., Xu, R., Chen, R., Li, S. S., Lu, S., Zhou, S., Chen, S., Wu, S., Ye, S., Ma, S., Wang, S., Zhou, S., Yu, S., Zhou, S., Zheng, S., Wang, T., Pei, T., Yuan, T., Sun, T., Xiao, W. L., Zeng, W., An, W., Liu, W., Liang, W., Gao, W., Zhang, W., Li, X. Q., Jin, X., Wang, X., Bi, X., Liu, X., Wang, X., Shen, X., Chen, X., Chen, X., Nie, X., Sun, X., Wang, X., Liu, X., Xie, X., Yu, X., Song, X., Zhou, X., Yang, X., Lu, X., Su, X., Wu, Y., Li, Y. K., Wei, Y. X., Zhu, Y. X., Xu, Y., Huang, Y., Li, Y., Zhao, Y., Sun, Y., Li, Y., Wang, Y., Zheng, Y., Zhang, Y., Xiong, Y., Zhao, Y., He, Y., Tang, Y., Piao, Y., Dong, Y., Tan, Y., Liu, Y., Wang, Y., Guo, Y., Zhu, Y., Wang, Y., Zou, Y., Zha, Y., Ma, Y., Yan, Y., You, Y., Liu, Y., Ren, Z. Z., Ren, Z., Sha, Z., Fu, Z., Huang, Z., Zhang, Z., Xie, Z., Hao, Z., Shao, Z., Wen, Z., Xu, Z., Zhang, Z., Li, Z., Wang, Z., Gu, Z., Li, Z., and Xie, Z. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, 2024. URL <https://arxiv.org/abs/2405.04434>.

Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, 2022. URL <https://arxiv.org/abs/2101.03961>.

Gupta, V., Koren, T., and Singer, Y. Shampoo: Pre-conditioned stochastic tensor optimization. 2018. URL <http://proceedings.mlr.press/v80/gupta18a/gupta18a.pdf>.

Kasieczka, G., Plehn, T., Thompson, J., and Russell, M. Top quark tagging reference dataset, March 2019.

Nakanishi, K. M. Scalable-softmax is superior for attention, 2025. URL <https://arxiv.org/abs/2501.19399>.

Qu, H., Li, C., and Qian, S. Particle transformer for jet tagging, 2024. URL <https://arxiv.org/abs/2202.03772>.

Vyas, N., Morwani, D., Zhao, R., Kwun, M., Shapira, I., Brandfonbrener, D., Janson, L., and Kakade, S. Soap: Improving and stabilizing shampoo using adam, 2025. URL <https://arxiv.org/abs/2409.11321>.

Wright, L. and Demeure, N. Ranger21: a synergistic deep learning optimizer, 2021. URL <https://arxiv.org/abs/2106.13731>.

Zhu, J., Chen, X., He, K., LeCun, Y., and Liu, Z. Transformers without normalization, 2025. URL <https://arxiv.org/abs/2503.10622>.